



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра прикладной математики (ПМ)**

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ
по дисциплине «Технологии и инструментарий анализа больших данных»

Практическая работа № 4

Студент группы *ИКБО-50-23, Враженко Д.О.*

(подпись)

Проверил *Старший преподаватель,
Воронцов Ю.А.*

(подпись)

Отчет представлен «___»_____2026 г.

Москва 2026 г.

Экспериментальный стенд и подготовка данных

Паспорт экспериментального стенда

Параметр	Фактическое значение
ОС	Windows-11-10.0.26300-SP0
Python	Python 3.14.0
Java	Temurin-17.0.20.1+1
PySpark / Spark	PySpark 4.2.0 / Spark 4.2.0
Логических CPU	12
Оперативная память	15,7
spark.master	local[*]
spark.default.parallelism	12
spark.sql.shuffle.partitions	8
spark.sql.adaptive.enabled	false

Проверка staging-каталога

```
geolocation          1000163 rows
order_items          112650 rows
products              32951 rows
sellers               3095 rows
product_category_name_translation  71 rows
```

Запуск SparkSession и контролируемая конфигурация

```
Master: local[*]
Default parallelism: 12
Shuffle partitions: 8
AQE: false
Spark UI: http://MSI:4040
```

Чтение исходных DataFrame и перевод в Parquet

```
1000163 112650 32951 3095
```

Spark UI как измерительный инструмент

Маркировка экспериментов в Spark UI

Details for Job 24

Status: SUCCEEDED
Submitted: 2026/10/05 17:54:41
Duration: 0.2 s
Associated SQL Query: 19
Job Group: P5_BASELINE
Completed Stages: 2

> Event Timeline
> DAG Visualization

✓ Completed Stages (2)

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
29	Практика 5: базовая проверка Spark UI count at NativeMethodAccessorImpl.java:0 +details	2026/10/05 17:54:41	16 ms	1/1			236.0 B	
28	Практика 5: базовая проверка Spark UI count at NativeMethodAccessorImpl.java:0 +details	2026/10/05 17:54:41	0.1 s	4/4	24.7 KiB			236.0 B

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

4.2.0 Started: 2026/10/05 17:51:16 Uptime: 0h 14m wrda

Исследование ленивых вычислений

Цепочка transformations без Action

DataFrame

Действие как момент запуска вычисления

Rows: 1000163

Elapsed: 0.488 s

Details for Job 25

Status: SUCCEEDED
Submitted: 2026/10/05 18:08:36
Duration: 0.3 s
Associated SQL Query: 20
Job Group: P5_LAZY
Completed Stages: 2

> Event Timeline
> DAG Visualization

✓ Completed Stages (2)

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
31	Lazy evaluation count at NativeMethodAccessorImpl.java:0 +details	2026/10/05 18:08:36	35 ms	1/1			708.0 B	
30	Lazy evaluation count at NativeMethodAccessorImpl.java:0 +details	2026/10/05 18:08:36	0.3 s	12/12	118.9 KiB			708.0 B

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

4.2.0 Started: 2026/10/05 18:08:10 Uptime: 0h 0m wrda

Исследование партиций и параллелизма

Число партиций и распределение строк

```
Input partitions: 12
Default parallelism: 12
+-----+
|pid|count|
+-----+
|0|90353|
|1|88373|
|2|88104|
|3|87668|
|4|86154|
|5|88355|
|6|87983|
|7|86853|
|8|86841|
|9|91182|
|10|92908|
|11|25389|
+-----+
```

spark

JobsStagesStorageEnvironmentExecutorsSQL / DataFrame

Practice5_SparkExecution application UI

Details for Job 26

Status: SUCCEEDED

Submitted: 2026/10/05 18:09:51

Duration: 0.5 s

Associated SQL Query: 21

Job Group: P5_LAZY

Completed Stages: 2

> Event Timeline

> DAG Visualization

Completed Stages (2)

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
33	Lazy evaluation showString at NativeMethodAccessorImpl.java:0	2026/10/05 18:09:51	81 ms	8/8			752.0 B	
32	Lazy evaluation showString at NativeMethodAccessorImpl.java:0	2026/10/05 18:09:51	0.4 s	12/12	73.3 KiB			752.0 B

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

4.2.0

Started: 2026/10/05 18:08:10

Uptime: 0h 3m

wrda

Реальный перекоc по geolocation_state

```
+-----+-----+
|geolocation_state|count|
+-----+-----+
|SP|404268|
|MG|126336|
|RJ|121169|
|RS|61851|
|PR|57859|
+-----+-----+
only showing top 5 rows
```

Сравнение repartition() и coalesce()

Контролируемое уменьшение партиций

skewed16: 16

repart4: 4

coal4: 4

Сравнение физического плана

```
=== repartition(4) ===
```

```
== Physical Plan ==
```

```
Exchange (3)
```

```
+-- * ColumnarToRow (2)
```

```
    +- Scan parquet (1)
```

```
(1) Scan parquet
```

```
Output [6]: [geolocation_id#141, geolocation_zip_code_prefix#142, geolocation_lat#143, geolocation_lng#144, geolocation_city#145, geolocation_state#146]
```

```
Batched: true
```

```
Location: InMemoryFileIndex [file:/C:/Additional/VirtualMachines/prac4/spark_input_parquet/geolocation]
```

```
ReadSchema: struct<geolocation_id:int,geolocation_zip_code_prefix:int,geolocation_lat:double,geolocation_lng:double,geolocation_city:string,geolocation_state:string>
```

```
(2) ColumnarToRow [codegen id : 1]
```

```
Input [6]: [geolocation_id#141, geolocation_zip_code_prefix#142, geolocation_lat#143, geolocation_lng#144, geolocation_city#145, geolocation_state#146]
```

```
(3) Exchange
```

```
Input [6]: [geolocation_id#141, geolocation_zip_code_prefix#142, geolocation_lat#143, geolocation_lng#144, geolocation_city#145, geolocation_state#146]
```

```
Arguments: RoundRobinPartitioning(4), REPARTITION_BY_NUM, [plan_id=466]
```

```
=== coalesce(4) ===
```

```
== Physical Plan ==
```

```
Coalesce (4)
```

```
+-- Exchange (3)
```

```
    +- * ColumnarToRow (2)
```

```
        +- Scan parquet (1)
```

```
(1) Scan parquet
```

```
Output [6]: [geolocation_id#141, geolocation_zip_code_prefix#142, geolocation_lat#143, geolocation_lng#144, geolocation_city#145, geolocation_state#146]
```

```
Batched: true
```

```
Location: InMemoryFileIndex [file:/C:/Additional/VirtualMachines/prac4/spark_input_parquet/geolocation]
```

```
ReadSchema: struct<geolocation_id:int,geolocation_zip_code_prefix:int,geolocation_lat:double,geolocation_lng:double,geolocation_city:string,geolocation_state:string>
```

```
(2) ColumnarToRow [codegen id : 1]
```

```
Input [6]: [geolocation_id#141, geolocation_zip_code_prefix#142, geolocation_lat#143, geolocation_lng#144, geolocation_city#145, geolocation_state#146]
```

```
(3) Exchange
```

```
Input [6]: [geolocation_id#141, geolocation_zip_code_prefix#142, geolocation_lat#143, geolocation_lng#144, geolocation_city#145, geolocation_state#146]
```

```
Arguments: hashpartitioning(geolocation_state#146, 16), REPARTITION_BY_NUM, [plan_id=489]
```

```
(4) Coalesce
```

```
Input [6]: [geolocation_id#141, geolocation_zip_code_prefix#142, geolocation_lat#143, geolocation_lng#144, geolocation_city#145, geolocation_state#146]
```

```
Arguments: 4
```

Размеры выходных партиций

```
repartition(4)
```

```
+---+-----+
|pid| count|
+---+-----+
|  0|250040|
|  1|250041|
|  2|250041|
|  3|250041|
+---+-----+
```

```
coalesce(4)
```

```
+---+-----+
|pid| count|
+---+-----+
|  0|195064|
|  1|626485|
|  2| 90152|
|  3| 88462|
+---+-----+
```

Вариант	min rows	max rows	mean rows	Комментарий
repartition(4)	250040	250041	250040.75	Полное перемешивание: строки распределены почти равномерно, $\min \approx \max \approx \text{mean} \approx 250041$
coalesce(4)	88462	626485	250040.75	Перемешивания нет: объединяет существующие партиции skewed16, унаследованный перекося сохраняется, $\min \ll \max$

Перемешивание данных на примере groupBy()

Агрегация order_items по продавцам

```
+-----+-----+-----+-----+
|seller_id|items|revenue|avg_price|
+-----+-----+-----+-----+
|4869f7a5dfa277a7dca6462dcf3b52b2|1156|229472.63|198.51|
|53243585a1d6dc2643021fd1853d8905|410|222776.05|543.36|
|4a3ca9315b744ce9f8e9374361493884|1987|200472.92|100.89|
|fa1c13f2614d7b5c4749cbc52fecda94|586|194042.03|331.13|
|7c67e1448b00f6e969d365cea6b010ab|1364|187923.89|137.77|
+-----+-----+-----+-----+
only showing top 5 rows
```

 [Jobs](#) [Stages](#) [Storage](#) [Environment](#) [Executors](#) [SQL / DataFrame](#)
Practice5_SparkExecution application UI

Details for Job 32

Status: SUCCEEDED
 Submitted: 2026/10/05 18:34:00
 Duration: 0.4 s
 Associated SQL Query: 27
 Job Group: P5_GROUPBY
 Completed Stages: 2

- > Event Timeline
- > DAG Visualization

Completed Stages (2)

Page: 1
1 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id ▾	Description		Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
53	Shuffle: groupBy seller_id showString at NativeMethodAccessorImpl.java:0	+details	2026/10/05 18:34:00	0.1 s	8/8			438.6 KiB	
52	Shuffle: groupBy seller_id showString at NativeMethodAccessorImpl.java:0	+details	2026/10/05 18:34:00	0.3 s	4/4	24.7 KiB			438.6 KiB

Page: 1
1 Pages. Jump to 1 . Show 100 items in a page. Go

4.2.0
Started: 2026/10/05 18:08:10
Uptime: 0h 26m


Анализ плана через explain("formatted")

Чтение плана снизу вверх

```
== Physical Plan ==
* HashAggregate (7)
+- Exchange (6)
   +- * HashAggregate (5)
      +- * Project (4)
         +- * Filter (3)
            +- * ColumnarToRow (2)
               +- Scan parquet (1)

(1) Scan parquet
Output [3]: [seller_id#150, price#152, freight_value#153]
Batched: true
Location: InMemoryFileIndex [file:/C:/Additional/VirtualMachines/prac4/spark_input_parquet/order_items]
PushedFilters: [IsNotNull(price), GreaterThanOrEqual(price,100.0)]
ReadSchema: struct<seller_id:string,price:double,freight_value:double>

(2) ColumnarToRow [codegen id : 1]
Input [3]: [seller_id#150, price#152, freight_value#153]

(3) Filter [codegen id : 1]
Input [3]: [seller_id#150, price#152, freight_value#153]
Condition : (isnotnull(price#152) AND (price#152 >= 100.0))

(4) Project [codegen id : 1]
Output [2]: [seller_id#150, (price#152 + freight_value#153) AS gross#398]
Input [3]: [seller_id#150, price#152, freight_value#153]

(5) HashAggregate [codegen id : 1]
Input [2]: [seller_id#150, gross#398]
Keys [1]: [seller_id#150]
Functions [1]: [partial_sum(gross#398)]
Aggregate Attributes [1]: [sum#405]
Results [2]: [seller_id#150, sum#406]

(6) Exchange
Input [2]: [seller_id#150, sum#406]
Arguments: hashpartitioning(seller_id#150, 8), ENSURE_REQUIREMENTS, [plan_id=958]

(7) HashAggregate [codegen id : 2]
Input [2]: [seller_id#150, sum#406]
Keys [1]: [seller_id#150]
Functions [1]: [sum(gross#398)]
Aggregate Attributes [1]: [sum(gross#398)#404]
Results [2]: [seller_id#150, round(sum(gross#398)#404, 2) AS gross_sum#399]
```

1) Scan parquet. Роль: чтение файлов.

- Источник – spark_input_parquet/order_items.

- Output [3] – используются только три столбца: seller_id, price,

freight_value. Остальные столбцы таблицы не читаются.

- PushedFilters: [IsNotNull(price), GreaterThanOrEqual(price, 100.0)] – предикат проталкивается в источник. Фильтр price >= 100 выполняется уже на уровне Parquet reader, что резко уменьшает объём прочитанных данных.

- Batched: true – чтение идёт пакетами.

2) ColumnarToRow. Роль: конвертация формата.

- Parquet читается в колоночном представлении, но дальнейшие операторы (Filter, Project, HashAggregate) работают со строчным представлением.

- ColumnarToRow – техническая граница между этими двумя мирами, не отдельный вычислительный шаг.

3) Filter. Роль: отбор строк.

- Condition: (isnotnull(price) AND (price >= 100.0))

- Физически Filter повторяет то, что уже было передано в Scan как PushedFilters. Строки, не прошедшие pushdown, отсекаются здесь. Для Parquet на этом шаге данных почти не остаётся – это подтверждается тем, что Filter и Scan в одном codegen id: 1.

4) Project. Роль: выбор/вычисление столбцов.

- Output [2]: [seller_id, (price + freight_value) AS gross]

- freight_value и price исчезли из выхода, остались только seller_id и новое поле gross. Это сокращает объём данных ещё до shuffle.

- gross = price + freight_value – вычисляется прямо здесь, в фазе Project.

Sort Merge Join и Broadcast Hash Join

Sort Merge Join при отключённом broadcast

```
== Physical Plan ==
* Project (12)
+- * SortMergeJoin Inner (11)
  :- * Sort (5)
    : +- Exchange (4)
    :   +- * Filter (3)
    :     +- * ColumnarToRow (2)
    :       +- Scan parquet (1)
  +- * Sort (10)
    +- Exchange (9)
      +- * Filter (8)
        +- * ColumnarToRow (7)
          +- Scan parquet (6)

(1) Scan parquet
Output [7]: [order_id#147, order_item_id#148, product_id#149, seller_id#150, shipping_limit_date#151, price#152, freight_value#153]
Batched: true
Location: InMemoryFileIndex [file:/C:/Additional/VirtualMachines/prac4/spark_input_parquet/order_items]
PushedFilters: [IsNotNull(product_id)]
ReadSchema: struct<order_id:string,order_item_id:int,product_id:string,seller_id:string,shipping_limit_date:timestamp,price:double,freight_value:double>
>

(2) ColumnarToRow [codegen id : 1]
Input [7]: [order_id#147, order_item_id#148, product_id#149, seller_id#150, shipping_limit_date#151, price#152, freight_value#153]

(3) Filter [codegen id : 1]
Input [7]: [order_id#147, order_item_id#148, product_id#149, seller_id#150, shipping_limit_date#151, price#152, freight_value#153]
Condition : isnotnull(product_id#149)

(4) Exchange
Input [7]: [order_id#147, order_item_id#148, product_id#149, seller_id#150, shipping_limit_date#151, price#152, freight_value#153]
Arguments: hashpartitioning(product_id#149, 8), ENSURE_REQUIREMENTS, [plan_id=1020]

(5) Sort [codegen id : 2]
Input [7]: [order_id#147, order_item_id#148, product_id#149, seller_id#150, shipping_limit_date#151, price#152, freight_value#153]
Arguments: [product_id#149 ASC NULLS FIRST], false, 0

(6) Scan parquet
Output [9]: [product_id#154, product_category_name#155, product_name_lenght#156, product_description_lenght#157, product_photos_qty#158, product_weight_g#159, product_length_cm#160, product_height_cm#161, product_width_cm#162]
Batched: true
Location: InMemoryFileIndex [file:/C:/Additional/VirtualMachines/prac4/spark_input_parquet/products]
PushedFilters: [IsNotNull(product_id)]
ReadSchema: struct<product_id:string,product_category_name:string,product_name_lenght:double,product_description_lenght:double,product_photos_qty:double,product_weight_g:double,product_length_cm:double,product_height_cm:double,product_width_cm:double>
>

(7) ColumnarToRow [codegen id : 3]
Input [9]: [product_id#154, product_category_name#155, product_name_lenght#156, product_description_lenght#157, product_photos_qty#158, product_weight_g#159, product_length_cm#160, product_height_cm#161, product_width_cm#162]

(8) Filter [codegen id : 3]
Input [9]: [product_id#154, product_category_name#155, product_name_lenght#156, product_description_lenght#157, product_photos_qty#158, product_weight_g#159, product_length_cm#160, product_height_cm#161, product_width_cm#162]
Condition : isnotnull(product_id#154)

(9) Exchange
Input [9]: [product_id#154, product_category_name#155, product_name_lenght#156, product_description_lenght#157, product_photos_qty#158, product_weight_g#159, product_length_cm#160, product_height_cm#161, product_width_cm#162]
Arguments: hashpartitioning(product_id#154, 8), ENSURE_REQUIREMENTS, [plan_id=1029]

(10) Sort [codegen id : 4]
Input [9]: [product_id#154, product_category_name#155, product_name_lenght#156, product_description_lenght#157, product_photos_qty#158, product_weight_g#159, product_length_cm#160, product_height_cm#161, product_width_cm#162]
Arguments: [product_id#154 ASC NULLS FIRST], false, 0

(11) SortMergeJoin [codegen id : 5]
Left keys [1]: [product_id#149]
Right keys [1]: [product_id#154]
Join type: Inner
Join condition: None

(12) Project [codegen id : 5]
Output [15]: [product_id#149, order_id#147, order_item_id#148, seller_id#150, shipping_limit_date#151, price#152, freight_value#153, product_category_name#155, product_name_lenght#156, product_description_lenght#157, product_photos_qty#158, product_weight_g#159, product_length_cm#160, product_height_cm#161, product_width_cm#162]
Input [16]: [order_id#147, order_item_id#148, product_id#149, seller_id#150, shipping_limit_date#151, price#152, freight_value#153, product_id#154, product_category_name#155, product_name_lenght#156, product_description_lenght#157, product_photos_qty#158, product_weight_g#159, product_length_cm#160, product_height_cm#161, product_width_cm#162]

Rows: 112650
```

Jobs |
 Stages |
 Storage |
 Environment |
 Executors |
 SQL / DataFrame

Practice5_SparkExecution application UI ▶

Details for Job 33

Status: SUCCEEDED
 Submitted: 2026/10/05 18:53:13
 Duration: 1 s
 Associated SQL Query: 28
 Job Group: P5_SMJ
 Completed Stages: 4

> Event Timeline
 > DAG Visualization

Completed Stages (4)

Page: 1
1 Pages. Jump to [1]. Show 100 items in a page. Go

Stage Id ▼	Description		Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
57	Sort Merge Join: items x products count at NativeMethodAccessorImpl.java:0	+details	2026/10/05 18:53:14	38 ms	1/1			472.0 B	
56	Sort Merge Join: items x products count at NativeMethodAccessorImpl.java:0	+details	2026/10/05 18:53:13	0.7 s	8/8			3.7 MiB	472.0 B
55	Sort Merge Join: items x products count at NativeMethodAccessorImpl.java:0	+details	2026/10/05 18:53:13	0.4 s	4/4	38.7 KiB			2.7 MiB
54	Sort Merge Join: items x products count at NativeMethodAccessorImpl.java:0	+details	2026/10/05 18:53:13	0.4 s	1/1	13.2 KiB			1088.3 KiB

Page: 1
1 Pages. Jump to [1]. Show 100 items in a page. Go

(9) Project [codegen id : 2]
Output [15]: [product_id#149, order_id#147, order_item_id#148, seller_id#150, shipping_limit_date#151, price#152, freight_value#153, product_category_name#155, product_name_lenght#156, product_description_lenght#157, product_photos_qty#158, product_weight_g#159, product_length_cm#160, product_height_cm#161, product_width_cm#162]
Input [16]: [order_id#147, order_item_id#148, product_id#149, seller_id#150, shipping_limit_date#151, price#152, freight_value#153, product_id#154, product_category_name#155, product_name_lenght#156, product_description_lenght#157, product_photos_qty#158, product_weight_g#159, product_length_cm#160, product_height_cm#161, product_width_cm#162]

Rows: 112650

Spark

JobsStagesStorageEnvironmentExecutorsSQL / DataFrame

Practice5_SparkExecution application UI

Details for Job 34

Status: SUCCEEDED
Submitted: 2026/10/05 18:56:18
Duration: 0.2 s
Associated SQL Query: 29
Job Group: P5_BHJ
Completed Stages: 1

> Event Timeline
> DAG Visualization

Completed Stages (1)

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
58	Broadcast Hash Join: items x products \$anonfun\$withThreadLocalCaptured\$1 at CompletableFuture.java:1768	2026/10/05 18:56:18	0.2 s	1/1	13.2 KiB			

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

4.2.0

Started: 2026/10/05 18:08:10

Uptime: 1h 6m

wrda

Spark

JobsStagesStorageEnvironmentExecutorsSQL / DataFrame

Practice5_SparkExecution application UI

Details for Job 35

Status: SUCCEEDED
Submitted: 2026/10/05 18:56:18
Duration: 0.2 s
Associated SQL Query: 29
Job Group: P5_BHJ
Completed Stages: 2

> Event Timeline
> DAG Visualization

Completed Stages (2)

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
60	Broadcast Hash Join: items x products count at NativeMethodAccessorImpl.java:0	2026/10/05 18:56:18	21 ms	1/1			236.0 B	
59	Broadcast Hash Join: items x products count at NativeMethodAccessorImpl.java:0	2026/10/05 18:56:18	0.1 s	4/4	38.7 KiB			236.0 B

Page: 1

1 Pages. Jump to 1. Show 100 items in a page. Go

4.2.0

Started: 2026/10/05 18:08:10

Uptime: 0h 49m

wrda

Сравнительная таблица JOIN-стратегий

Показатель	Sort Merge Join	Broadcast Hash Join
Строк результата	112 650	112 650
Exchange большой стороны	1	0
Shuffle Read / Write	3.7 MiB / 3.76 MiB	236.0 B / 236.0 B
Jobs / Stages	1 / 4	2 / 3
Время, с	1	0,4

Исследование cache() / persist()

Повторно используемый промежуточный DataFrame

Измерение без persist()

```
Action 1: 1.230 s
Action 2: 0.808 s
is_cached: False
```

Сохранение persist(MEMORY_AND_DISK) и материализация

```
После persist: True
Rows: 112650
Materialization: 1.302 s
Cached action 1: 0.440 s
Cached action 2: 0.248 s
После unpersist: False
```

Вариант	Подготовка/материализация, с	Action 1, с	Action 2, с
Без persist	—	1.230	0.808
MEMORY_AND_DISK	1.302	0.440	0.248

Влияние числа партиций на время выполнения

Описание и код контролируемого бенчмарка

```
1 [0.21919090000301367, 0.2042189000021608, 0.16805439999734517] 0.2042189000021608 7453561042.825421
2 [0.7624811999994563, 0.452472700002545, 0.25637960000312887] 0.452472700002545 7453561042.82542
4 [0.5755437000007078, 0.566678300001513, 0.5787076999986311] 0.5755437000007078 7453561042.825421
8 [0.8182559000015317, 0.3911566999995557, 0.6140071999980137] 0.6140071999980137 7453561042.825422
16 [1.1094055999965349, 0.5379385000014736, 0.885352099998272] 0.885352099998272 7453561042.82542
32 [2.144602399997124, 1.3569344999996247, 1.5322424000005412] 1.5322424000005412 7453561042.825421
```

Таблица и относительное ускорение

p	Run 1, с	Run 2, с	Run 3, с	Median, с	S(p)=T(1)/T(p)
1	0,2192	0,2042	0,1681	0,2042	1,00
2	0,7625	0,4525	0,2564	0,4525	0,4513
4	0,5755	0,5667	0,5787	0,5755	0,3548
8	0,8183	0,3912	0,6140	0,6140	0,3326
16	1,1094	0,5379	0,8854	0,8854	0,2306
32	2,1446	1,3569	1,5322	1,5322	0,1333

Дополнительный опыт с перемешиванием партиций

```
q= 2; groups=27; time=1.728s
q= 4; groups=27; time=1.226s
q= 8; groups=27; time=1.149s
q=16; groups=27; time=1.265s
q=32; groups=27; time=1.624s
```

Партиции и маленькие файлы

Запись одного DataFrame с разным числом партиций

```
1 (1, 6.456236839294434, 6611.1865234375)
4 (4, 7.432483673095703, 1902.7158203125)
16 (16, 8.732264518737793, 558.8649291992188)
64 (64, 10.285961151123047, 164.57537841796875)
```

p	part-файлов	Общий размер, MiB	Средний размер, KiB
1	1	6,4562	6611,1865
4	4	7,4325	1902,7158
16	16	8,7323	558,8649
64	64	10,2860	164,5754

Почему coalesce(1) не является универсальным решением

```
Partitions before write: 1
```

Дополнительный эксперимент: Adaptive Query Execution

Повтор groupBy() с AQE

```
== Physical Plan ==
AdaptiveSparkPlan (5)
+- HashAggregate (4)
   +- Exchange (3)
      +- HashAggregate (2)
         +- Scan parquet (1)

(1) Scan parquet
Output [1]: [geolocation_state#146]
Batched: true
Location: InMemoryFileIndex [file:/C:/Additional/VirtualMachines/prac4/spark_input_parquet/geolocation]
ReadSchema: struct<geolocation_state:string>

(2) HashAggregate
Input [1]: [geolocation_state#146]
Keys [1]: [geolocation_state#146]
Functions [1]: [partial_count(1)]
Aggregate Attributes [1]: [count#2173L]
Results [2]: [geolocation_state#146, count#2174L]

(3) Exchange
Input [2]: [geolocation_state#146, count#2174L]
Arguments: hashpartitioning(geolocation_state#146, 32), ENSURE_REQUIREMENTS, [plan_id=4154]

(4) HashAggregate
Input [2]: [geolocation_state#146, count#2174L]
Keys [1]: [geolocation_state#146]
Functions [1]: [count(1)]
Aggregate Attributes [1]: [count(1)#2172L]
Results [2]: [geolocation_state#146, count(1)#2172L AS count#2165L]
```

```
(5) AdaptiveSparkPlan
Output [2]: [geolocation_state#146, count#2165L]
Arguments: isFinalPlan=false
```

AQE: true

Итоговый анализ Spark UI

Эксперимент	Jobs	Stages	Tasks в ключевой Stage	Shuffle Write	Shuffle Read	Что подтверждает UI
groupBy seller_id	1	2	8	438.6 KiB	438.6 KiB	Граница стадий = Exchange hashpartitioning(seller_id, 8). Stage 52 (4 задачи) – partial HashAggregate + Shuffle Write; Stage 53 (8 задач) – final HashAggregate + Shuffle Read.
repartition(4)	1	5	4	248.0 B	20.7 KiB	Stage 45 содержит 4 задачи – ровно столько, сколько задано в repartition(4). Ненулевые Shuffle Read/Write подтверждают полное перемешивание (Exchange RoundRobinPartitioning(4)), а не Coalesce.
Sort Merge Join	1	4	8	3.8 MiB	3.7 MiB	Два Shuffle Write – по одному на каждую сторону JOIN. Stage 56 (8 задач, Shuffle Read 3.7 MiB) – SortMergeJoin. Это доказывает, что SMJ перемешивает обе стороны.
Broadcast Hash Join	1	2	4	236.0 B	236.0 B	У Stage 59 (BroadcastHashJoin) Shuffle Write ≈ 0, у Stage 60 Shuffle Read ≈ 0. Большая сторона (order_items, 4 партии) не перемешивается. 236 B – технический след финального count().
persist + повторный Action	4	3	8	387.6 KiB	387.6 KiB	Job 96 содержит Skipped Stage 213 – материализованный DataFrame читается из кэша, повторный расчёт пропущен.

Формирование экспериментальных выводов

1. Lazy evaluation подтверждён экспериментально

- Контекст: geolocation (1 000 163 строки, 12 партиций Parquet),

spark.sql.shuffle.partitions=8, AQE=false, local[*], 12 логических CPU.

- Что делали: построили geo_prepared = geo.select(...).filter(...).withColumn(...), затем вызвали count().

- Наблюдение: после построения DataFrame в Spark UI не появилось новых Job; только count() инициировал Job P5_LAZY_ACTION (2 стадии, 13 задач). Число строк результата – 1 000 163: фильтр geolocation_state IS NOT NULL не отбросил ни одной строки, потому что NULL в этом столбце отсутствуют.

- Вывод: без Action Spark не выполняет вычислений, а только строит план. Замер времени вокруг цепочки преобразований без Action измерял бы работу драйвера, а не обработку данных.

2. Одна задача = одна партиция

- Контекст: geolocation после CSV→Parquet имеет 12 партиций (по числу записанных чанков), defaultParallelism = 12.

- Что делали: partition_sizes = geo.select(spark_partition_id()).groupBy("pid").count().

- Наблюдение: в Spark UI первая стадия Job P5_LAZY_ACTION содержит 12 Tasks. Число задач совпадает с числом партиций, а не с числом вычислительных слотов.

- Вывод: модель «одна задача обрабатывает одну партицию» подтверждается напрямую. Если число партиций меньше числа слотов (например, products имеет 1 партицию, а слотов 12), часть CPU остаётся незадействованной.

3. Двухфазная агрегация со shuffle при groupBy

- Контекст: order_items (112 650 строк, 4 партиции), groupBy("seller_id") (блок 8).

- Что делали: seller_stats = items.groupBy("seller_id").agg(count, sum, avg).

- Наблюдение: в плане – partial HashAggregate → Exchange hashpartitioning(seller_id, 8) → final HashAggregate. В Spark UI Job P5_GROUPBY содержит 2 стадии: Stage 52 (4 задачи, Shuffle Write 438.6 KiB) и Stage 53 (8 задач, Shuffle Read 438.6 KiB). Число 8 = spark.sql.shuffle.partitions.

- Вывод: Catalyst оптимизирует агрегацию в две фазы: сначала суммирует локально внутри партиции, и только частичные результаты уходят на shuffle.

Это сокращает объём shuffle-данных с размера исходных строк до размера промежуточных сумм.

4. repartition() делает shuffle, coalesce() – нет

- Контекст: geo (12 партиций), skewed16 = geo.repartition(16, "geolocation_state") – перекоc SP $\approx$ 40 %.

- Что делали: repart4 = skewed16.repartition(4), coal4 = skewed16.coalesce(4) (блок 7).

- Наблюдение: explain("formatted") показывает Exchange RoundRobinPartitioning(4) у repart4 и Coalesce 4 у coal4. В Spark UI Job P5_TABLE4 для repartition(4) содержит Stage 45 с 4 задачами и Shuffle Read 20.7 KiB / Write 248.0 B. У coalesce(4) в плане нет Exchange. Распределение строк: у repartition близко к равномерному, у coalesce сохраняется перекоc от geolocation_state (min $\ll$ max).

- Вывод: repartition(n) выполняет полное перемешивание, но даёт равномерное распределение; coalesce(n) дешевле, но наследует перекоc исходного разбиения. Равное число выходных партиций (4) не означает равную стоимость.

5. Sort Merge Join перемешивает обе стороны

- Контекст: order_items $\bowtie$ products по product_id, hint("merge"), autoBroadcastJoinThreshold=-1.

- Что делали: items.hint("merge").join(products.hint("merge"), "product_id").

- Наблюдение: в плане – по Exchange hashpartitioning(product_id, 8) и Sort на каждой стороне, затем SortMergeJoin. В Spark UI Job P5_SMJ содержит 4 стадии: Stage 54 (1 задача, Shuffle Write 1088.3 KiB по products) + Stage 55 (4 задачи, Shuffle Write 2.7 MiB по order_items) $\rightarrow$ Stage 56 (SortMergeJoin, 8 задач, Shuffle Read 3.7 MiB) $\rightarrow$ Stage 57 (1 задача, финальный count). Результат – 112 650 строк.

- Вывод: SMJ требует привести обе стороны к одинаковому распределению по ключу JOIN. Обе стороны проходят shuffle и sort, что даёт 2 $\times$ Shuffle Write и 1 $\times$ Shuffle Read.

6. Broadcast Hash Join не перемешивает большую сторону

- Контекст: тот же order_items $\bowtie$ products, но правая сторона обёрнута в

F.broadcast().

- Что делали: `items.join(F.broadcast(products), "product_id", "inner")`.
- Наблюдение: в плане – BroadcastExchange HashedRelationBroadcastMode для products и BroadcastHashJoin без Exchange для order_items. Job P5_BHJ содержит 2 стадии: Stage 59 (4 задачи, Shuffle Write ≈ 0) → Stage 60 (1 задача, Shuffle Read ≈ 0). Итог – те же 112 650 строк.
- Вывод: при одинаковом логическом результате физический план отличается принципиально. ВНЖ копирует маленькую таблицу всем исполнителям, и большая сторона не проходит shuffle. Это радикально снижает стоимость – но только пока products действительно мал (32 951 строк, ~5–10 МБ).

7. persist() материализует промежуточный результат

- Контекст: `joined = items ⋈ broadcast(products_small) ⋈ broadcast(sellers_small)`, два независимых действия `action_category` и `action_state`.
- Что делали: `cached = joined.persist(MEMORY_AND_DISK)`, затем `cached.count()` (материализация) и повторные Actions.
- Наблюдение: в Spark UI Job 84 (без кэша) читает всю цепочку заново. Job 96 содержит Skipped Stage 213 – Spark распознал, что данные уже в кэше, и пропустил пересчёт. Замеры Materialization / Cached action 1 / Cached action 2 показывают, что первый прогон стоит столько же, сколько без кэша, а повторные – заметно дешевле и без Shuffle.
- Вывод: кэш окупается только при повторном использовании. При однократном действии `persist()` даёт отрицательный эффект – вы платите за материализацию без выигрыша. `unpersist(blocking=True)` освобождает место.

8. Зависимость времени от p – компромисс, а не монотонность

- Контекст: `spark.range(0, 5_000_000, numPartitions=p), withColumn(id%1000, sqrt(id+1)), groupBy(key).agg(sum), shuffle.partitions=8`.
- Что делали: $p \in \{1, 2, 4, 8, 16, 32\}$, по 3 замера, медиана.
- Наблюдение: $T(1)=0.204$ с, $T(2)=0.453$, $T(4)=0.576$, $T(8)=0.614$, $T(16)=0.885$, $T(32)=1.532$. Относительное ускорение $S(p) = T(1)/T(p)$ монотонно убывает ($1.00 \rightarrow 0.13$). Ожидаемого пика на $p \approx 4-8$ нет.
- Вывод: при тривиальной вычислительной нагрузке увеличение p не даёт ускорения – растут только накладные расходы планировщика и число shuffle-

файлов. Это не противоречие теории, а подтверждение тезиса о компромиссе: оптимум p зависит от соотношения полезной работы на задачу и overhead. Для получения классической U-кривой нужно либо усложнить вычисление, либо увеличить N до размеров, где CPU-нагрузка доминирует.

Эксперимент над другой связкой таблиц ecommerce

Вариант	Основная таблица	Справочник / вторая таблица	Ключ	Пример группировки
A	orders	customers	customer_id	customer_state
1. Прочитать выбранные таблицы из Parquet и зафиксировать число строк и исходное число partitions. 2. Сформировать цепочку минимум из трёх transformations и экспериментально показать lazy evaluation до и после Action. 3. Провести один опыт repartition()/coalesce() и сравнить physical plan двух вариантов. 4. Выполнить groupBy() по выбранному категориальному ключу, найти Exchange и зафиксировать Shuffle Read/Write во Spark UI. 5. Выполнить один и тот же JOIN как Sort Merge Join и Broadcast Hash Join. Сравнить планы, число строк результата, Stages и время. 6. Выбрать промежуточный DataFrame, используемый минимум двумя Actions, и проверить эффект persist(MEMORY_AND_DISK). 7. Провести серию минимум из четырёх значений partitions; для каждого сделать не менее трёх повторов и использовать медиану. 8. Записать итоговый DataFrame минимум с двумя разными числами partitions и сопоставить их с количеством part-файлов. 9. Приложить к отчёту не менее трёх фрагментов Spark UI: Stages/Tasks, SQL/DataFrame plan и Storage либо Join stage. 10. Сформулировать не менее пяти собственных экспериментальных выводов и минимум два ограничения измерений.				

```
from pathlib import Path
import os
```

```
PROJECT_ROOT = Path(r"C:\Additional\VirtualMachines\prac4")
os.chdir(PROJECT_ROOT)
assert PROJECT_ROOT.exists(), PROJECT_ROOT
print("CWD:", Path.cwd())
```

CWD: C:\Additional\VirtualMachines\prac4

```
from pyspark.sql import SparkSession, functions as F
from pyspark import StorageLevel
from time import perf_counter
from statistics import median
```

```
spark = (
    SparkSession.builder
        .master("local[*]")
        .config("spark.sql.shuffle.partitions", "8")
        .config("spark.sql.adaptive.enabled", "false")
        .config("spark.sql.autoBroadcastJoinThreshold", "-1")
        .getOrCreate()
)
```

```
spark.sparkContext.setLogLevel("WARN")
sc = spark.sparkContext
```

```
print("Master:", sc.master)
print("Default parallelism:", sc.defaultParallelism)
print("Shuffle partitions:", spark.conf.get("spark.sql.shuffle.partitions"))
print("AQE:", spark.conf.get("spark.sql.adaptive.enabled"))
print("Spark UI:", sc.uiWebViewUrl)
```

C:\Additional\VirtualMachines\prac4\.venv\Lib\site-packages\pyspark\testing\utils.py:127: FutureWarning: PySpark does not yet fully support pandas >= 3.0.0. Some features may not work correctly. It is recommended to use pandas < 3.0.0 for now.

```
require_minimum_pandas_version()
```

```
Master: local[*]
Default parallelism: 12
Shuffle partitions: 8
AQE: false
Spark UI: http://MSI:4040
```

```
import sqlite3
import pandas as pd
```

```
DB_PATH = Path("ecommerce.sqlite")
STAGING = Path("spark_input")
PARQUET = Path("spark_input_parquet")
STAGING.mkdir(exist_ok=True)
PARQUET.mkdir(exist_ok=True)
```

```
NEW_TABLES = ["orders", "customers"]
```

```
with sqlite3.connect(DB_PATH) as con:
    existing = {r[0] for r in con.execute(
        "SELECT name FROM sqlite_master WHERE type='table'")}
print("Таблицы в SQLite:", sorted(existing))
```

```
for table in NEW_TABLES:
    if table not in existing:
        print(f"⚠ {table} отсутствует в SQLite – пропускаем")
        continue
```

```
csv_path = STAGING / f"{table}.csv"
if not csv_path.exists():
    first, total = True, 0
    with sqlite3.connect(DB_PATH) as con:
        for chunk in pd.read_sql_query(f"SELECT * FROM {table}",
                                         con, chunksize=100_000):
            chunk.to_csv(csv_path, index=False,
                         mode="w" if first else "a",
                         header=first, encoding="utf-8")
            total += len(chunk)
            first = False
```

```

    print(f"{table}: {total} rows → CSV")
else:
    print(f"{table}: CSV уже существует")

df = (spark.read
      .option("header", True)
      .option("inferSchema", True)
      .csv(str(csv_path)))
df.write.mode("overwrite").parquet(str(PARQUET / table))
print(f"{table}: Parquet записан в {PARQUET / table}")

```

Таблицы в SQLite: ['customers', 'geolocation', 'leads_closed', 'leads_qualified', 'order_items', 'order_payments', 'order_reviews', 'orders', 'product_category_name_translation', 'products', 'sellers', 'sqlite_sequence', 'sqlite_stat1', 'sqlite_stat4']

orders: CSV уже существует

orders: Parquet записан в spark_input_parquet/orders

customers: CSV уже существует

customers: Parquet записан в spark_input_parquet/customers

1. Прочитать выбранные таблицы из Parquet и зафиксировать число строк и исходное число partitions.

```

orders = spark.read.parquet(str(PARQUET / "orders"))
customers = spark.read.parquet(str(PARQUET / "customers"))

```

```
orders_rows = orders.count()
```

```
customers_rows = customers.count()
```

```

print("\norders:")
print(f"rows = {orders_rows}")
print(f"partitions = {orders.rdd.getNumPartitions()}")
print("\norders schema:")
orders.printSchema()

```

```

print("\ncustomers:")
print(f"rows = {customers_rows}")
print(f"partitions = {customers.rdd.getNumPartitions()}")
print("\ncustomers schema:")
customers.printSchema()

```

orders:

rows = 99441

partitions = 5

orders schema:

root

```

|-- order_id: string (nullable = true)
|-- customer_id: string (nullable = true)
|-- order_status: string (nullable = true)
|-- order_purchase_timestamp: timestamp (nullable = true)
|-- order_approved_at: timestamp (nullable = true)

```

```
|-- order_delivered_carrier_date: timestamp (nullable = true)
|-- order_delivered_customer_date: timestamp (nullable = true)
|-- order_estimated_delivery_date: timestamp (nullable = true)
```

```
customers:
rows = 99441
partitions = 3
```

```
customers schema:
root
|-- customer_id: string (nullable = true)
|-- customer_unique_id: string (nullable = true)
|-- customer_zip_code_prefix: integer (nullable = true)
|-- customer_city: string (nullable = true)
|-- customer_state: string (nullable = true)
```

2. Сформировать цепочку минимум из трёх transformations и экспериментально показать lazy evaluation до и после Action.

```
sc.setJobGroup("SS_LAZY", "Lazy evaluation – no action")
```

```
orders_prepared = (
    orders
    .select("order_id", "customer_id", "order_status", "order_purchase_timestamp")
    .filter(F.col("order_status").isNotNull())
    .withColumn("purchase_year", F.year("order_purchase_timestamp"))
    .withColumn("is_delivered", (F.col("order_status") == "delivered").cast("int"))
)
```

```
print("Тип объекта:", type(orders_prepared).__name__)
```

Тип объекта: DataFrame

```
sc.setJobGroup("SS_LAZY_ACTION", "Lazy evaluation – action")
```

```
t0 = perf_counter()
n = orders_prepared.count()
t1 = perf_counter()
```

```
print("Rows:", n)
print(f"Elapsed: {t1 - t0:.3f} s")
```

```
Rows: 99441
Elapsed: 0.513 s
```

3. Провести один опыт repartition()/coalesce() и сравнить physical plan двух вариантов.

```
skewed = orders.repartition(16, "order_status")
repart4 = skewed.repartition(4)
```

```

coal4    = skewed.coalesce(4)

print("skewed:", skewed.rdd.getNumPartitions())
print("repart4:", repart4.rdd.getNumPartitions())
print("coal4:", coal4.rdd.getNumPartitions())

skewed: 16
repart4: 4
coal4: 4

print("==== repartition(4) =====")
repart4.explain("formatted")

print("\n===== coalesce(4) =====")
coal4.explain("formatted")

===== repartition(4) =====
== Physical Plan ==
Exchange (3)
+- * ColumnarToRow (2)
   +- Scan parquet (1)

(1) Scan parquet
Output [8]: [order_id#60, customer_id#61, order_status#62, or-
der_purchase_timestamp#63, order_approved_at#64, or-
der_delivered_carrier_date#65, order_delivered_customer_date#66, or-
der_estimated_delivery_date#67]
Batched: true
Location: InMemoryFileIndex
[file:/C:/Additional/VirtualMachines/prac4/spark_input_parquet/orders]
ReadSchema:
struct<order_id:string,customer_id:string,order_status:string,order_purchase_tim-
es-
tamp:timestamp,order_approved_at:timestamp,order_delivered_carrier_date:timestamp,
p,order_delivered_customer_date:timestamp,order_estimated_delivery_date:timestamp
p>

(2) ColumnarToRow [codegen id : 1]
Input [8]: [order_id#60, customer_id#61, order_status#62, or-
der_purchase_timestamp#63, order_approved_at#64, or-
der_delivered_carrier_date#65, order_delivered_customer_date#66, or-
der_estimated_delivery_date#67]

(3) Exchange
Input [8]: [order_id#60, customer_id#61, order_status#62, or-
der_purchase_timestamp#63, order_approved_at#64, or-
der_delivered_carrier_date#65, order_delivered_customer_date#66, or-
der_estimated_delivery_date#67]
Arguments: RoundRobinPartitioning(4), REPARTITION_BY_NUM, [plan_id=209]

```

```

===== coalesce(4) =====
== Physical Plan ==
Coalesce (4)
+- Exchange (3)
   +- * ColumnarToRow (2)
      +- Scan parquet (1)

```

(1) Scan parquet

```

Output [8]: [order_id#60, customer_id#61, order_status#62, or-
der_purchase_timestamp#63, order_approved_at#64, or-
der_delivered_carrier_date#65, order_delivered_customer_date#66, or-
der_estimated_delivery_date#67]
Batched: true
Location: InMemoryFileIndex
[file:/C:/Additional/VirtualMachines/prac4/spark_input_parquet/orders]
ReadSchema:
struct<order_id:string,customer_id:string,order_status:string,order_purchase_tim-
es-
tamp:timestamp,order_approved_at:timestamp,order_delivered_carrier_date:timeスタ-
mp,order_delivered_customer_date:timestamp,order_estimated_delivery_date:timeスタ-
mp>

```

(2) ColumnarToRow [codegen id : 1]

```

Input [8]: [order_id#60, customer_id#61, order_status#62, or-
der_purchase_timestamp#63, order_approved_at#64, or-
der_delivered_carrier_date#65, order_delivered_customer_date#66, or-
der_estimated_delivery_date#67]

```

(3) Exchange

```

Input [8]: [order_id#60, customer_id#61, order_status#62, or-
der_purchase_timestamp#63, order_approved_at#64, or-
der_delivered_carrier_date#65, order_delivered_customer_date#66, or-
der_estimated_delivery_date#67]
Arguments: hashpartitioning(order_status#62, 16), REPARTITION_BY_NUM,
[plan_id=232]

```

(4) Coalesce

```

Input [8]: [order_id#60, customer_id#61, order_status#62, or-
der_purchase_timestamp#63, order_approved_at#64, or-
der_delivered_carrier_date#65, order_delivered_customer_date#66, or-
der_estimated_delivery_date#67]
Arguments: 4

```

```

def partition_profile(df):
    return (df.select(F.spark_partition_id().alias("pid"))
            .groupBy("pid").count().orderBy("pid"))

```

```

sc.setJobGroup("SS_TASK3", "repartition vs coalesce – profiles")

```

```

print("repartition(4):")
repart_profile = partition_profile(repart4).collect()
for row in repart_profile:
    print(f"pid={row['pid']} count={row['count']}")

print("\ncoalesce(4):")
coal_profile = partition_profile(coal4).collect()
for row in coal_profile:
    print(f"pid={row['pid']} count={row['count']}")

repartition(4):
pid=0 count=24860
pid=1 count=24862
pid=2 count=24859
pid=3 count=24860

coalesce(4):
pid=0 count=615
pid=1 count=1721
pid=2 count=96480
pid=3 count=625

def partition_stats(profile_rows, label):
    counts = [r['count'] for r in profile_rows]
    print(f"{label}: min={min(counts)} max={max(counts)} "
          f"mean={sum(counts)/len(counts):.0f} total={sum(counts)}")

partition_stats(repart_profile, "repartition(4)")
partition_stats(coal_profile, "coalesce(4)")

repartition(4): min=24859 max=24862 mean=24860 total=99441
coalesce(4): min=615 max=96480 mean=24860 total=99441

```

- *repartition(4)* в плане → *Exchange RoundRobinPartitioning(4)*, есть полное перемешивание.
- *coalesce(4)* в плане → *Coalesce 4*, перемешивания нет.
- Оба DataFrame имеют 4 партии и одинаковое число строк (99 441), но:
 - *repartition(4)* распределяет строки близко к равномерно;
 - *coalesce(4)* сохраняет исходный перекоп, полученный после *repartition(16, "order_status")*.

4. Выполнить `groupBy()` по выбранному категориальному ключу, найти *Exchange* и зафиксировать *Shuffle Read/Write* во Spark UI.

```

sc.setJobGroup("SS_TASK4", "groupBy customer_state")

state_stats = (
    customers
    .groupBy("customer_state")
    .agg(F.count("*").alias("customers"))
    .orderBy(F.desc("customers"))
)

```

```
state_stats.show(30, truncate=False)
```

```
+-----+-----+
|customer_state|customers|
+-----+-----+
|SP            |41746   |
|RJ            |12852   |
|MG            |11635   |
|RS            |5466    |
|PR            |5045    |
|SC            |3637    |
|BA            |3380    |
|DF            |2140    |
|ES            |2033    |
|GO            |2020    |
|PE            |1652    |
|CE            |1336    |
|PA            |975     |
|MT            |907     |
|MA            |747     |
|MS            |715     |
|PB            |536     |
|PI            |495     |
|RN            |485     |
|AL            |413     |
|SE            |350     |
|TO            |280     |
|RO            |253     |
|AM            |148     |
|AC            |81      |
|AP            |68      |
|RR            |46      |
+-----+-----+
```

```
state_stats.explain("formatted")
```

```
== Physical Plan ==
* Sort (7)
+- Exchange (6)
  +- * HashAggregate (5)
    +- Exchange (4)
      +- * HashAggregate (3)
        +- * ColumnarToRow (2)
          +- Scan parquet (1)
```

```
(1) Scan parquet
Output [1]: [customer_state#72]
Batched: true
Location: InMemoryFileIndex
[file:/C:/Additional/VirtualMachines/prac4/spark_input_parquet/customers]
```

ReadSchema: struct<customer_state:string>

(2) ColumnarToRow [codegen id : 1]

Input [1]: [customer_state#72]

(3) HashAggregate [codegen id : 1]

Input [1]: [customer_state#72]

Keys [1]: [customer_state#72]

Functions [1]: [partial_count(1)]

Aggregate Attributes [1]: [count#129L]

Results [2]: [customer_state#72, count#130L]

(4) Exchange

Input [2]: [customer_state#72, count#130L]

Arguments: hashpartitioning(customer_state#72, 8), ENSURE_REQUIREMENTS, [plan_id=485]

(5) HashAggregate [codegen id : 2]

Input [2]: [customer_state#72, count#130L]

Keys [1]: [customer_state#72]

Functions [1]: [count(1)]

Aggregate Attributes [1]: [count(1)#125L]

Results [2]: [customer_state#72, count(1)#125L AS customers#119L]

(6) Exchange

Input [2]: [customer_state#72, customers#119L]

Arguments: rangepartitioning(customers#119L DESC NULLS LAST, 8), ENSURE_REQUIREMENTS, [plan_id=489]

(7) Sort [codegen id : 3]

Input [2]: [customer_state#72, customers#119L]

Arguments: [customers#119L DESC NULLS LAST], true, 0

5. Выполнить один и тот же JOIN как Sort Merge Join и Broadcast Hash Join.

Сравнить планы, число строк результата, Stages и время.

```
sc.setJobGroup("SS_SMJ", "Sort Merge Join: orders x customers")
```

```
smj = (
    orders.hint("merge")
        .join(customers.hint("merge"), on="customer_id", how="inner")
)
```

```
smj.explain("formatted")
print("Строк в результате:", smj.count())
```

```
== Physical Plan ==
* Project (12)
+- * SortMergeJoin Inner (11)
   :- * Sort (5)
   : +- Exchange (4)
```

```

:      +- * Filter (3)
:      +- * ColumnarToRow (2)
:      +- Scan parquet (1)
+- * Sort (10)
    +- Exchange (9)
        +- * Filter (8)
            +- * ColumnarToRow (7)
                +- Scan parquet (6)

```

(1) Scan parquet

Output [8]: [order_id#60, customer_id#61, order_status#62, order_purchase_timestamp#63, order_approved_at#64, order_delivered_carrier_date#65, order_delivered_customer_date#66, order_estimated_delivery_date#67]

Batched: true

Location: InMemoryFileIndex

[file:/C:/Additional/VirtualMachines/prac4/spark_input_parquet/orders]

PushedFilters: [IsNotNull(customer_id)]

ReadSchema:

struct<order_id:string,customer_id:string,order_status:string,order_purchase_timestamp:

timestamp,order_approved_at:timestamp,order_delivered_carrier_date:timestamp,order_delivered_customer_date:timestamp,order_estimated_delivery_date:timestamp>

(2) ColumnarToRow [codegen id : 1]

Input [8]: [order_id#60, customer_id#61, order_status#62, order_purchase_timestamp#63, order_approved_at#64, order_delivered_carrier_date#65, order_delivered_customer_date#66, order_estimated_delivery_date#67]

(3) Filter [codegen id : 1]

Input [8]: [order_id#60, customer_id#61, order_status#62, order_purchase_timestamp#63, order_approved_at#64, order_delivered_carrier_date#65, order_delivered_customer_date#66, order_estimated_delivery_date#67]

Condition : isnotnull(customer_id#61)

(4) Exchange

Input [8]: [order_id#60, customer_id#61, order_status#62, order_purchase_timestamp#63, order_approved_at#64, order_delivered_carrier_date#65, order_delivered_customer_date#66, order_estimated_delivery_date#67]

Arguments: hashpartitioning(customer_id#61, 8), ENSURE_REQUIREMENTS, [plan_id=553]

(5) Sort [codegen id : 2]

Input [8]: [order_id#60, customer_id#61, order_status#62, order_purchase_timestamp#63, order_approved_at#64, order_delivered_carrier_date#65, order_delivered_customer_date#66, order_estimated_delivery_date#67]

Arguments: [customer_id#61 ASC NULLS FIRST], false, 0

(6) Scan parquet

Output [5]: [customer_id#68, customer_unique_id#69, customer_zip_code_prefix#70, customer_city#71, customer_state#72]

Batched: true

Location: InMemoryFileIndex

[file:/C:/Additional/VirtualMachines/prac4/spark_input_parquet/customers]

PushedFilters: [IsNotNull(customer_id)]

ReadSchema:

struct<customer_id:string,customer_unique_id:string,customer_zip_code_prefix:int,customer_city:string,customer_state:string>

(7) ColumnarToRow [codegen id : 3]

Input [5]: [customer_id#68, customer_unique_id#69, customer_zip_code_prefix#70, customer_city#71, customer_state#72]

(8) Filter [codegen id : 3]

Input [5]: [customer_id#68, customer_unique_id#69, customer_zip_code_prefix#70, customer_city#71, customer_state#72]

Condition : isnotnull(customer_id#68)

(9) Exchange

Input [5]: [customer_id#68, customer_unique_id#69, customer_zip_code_prefix#70, customer_city#71, customer_state#72]

Arguments: hashpartitioning(customer_id#68, 8), ENSURE_REQUIREMENTS, [plan_id=562]

(10) Sort [codegen id : 4]

Input [5]: [customer_id#68, customer_unique_id#69, customer_zip_code_prefix#70, customer_city#71, customer_state#72]

Arguments: [customer_id#68 ASC NULLS FIRST], false, 0

(11) SortMergeJoin [codegen id : 5]

Left keys [1]: [customer_id#61]

Right keys [1]: [customer_id#68]

Join type: Inner

Join condition: None

(12) Project [codegen id : 5]

Output [12]: [customer_id#61, order_id#60, order_status#62, order_purchase_timestamp#63, order_approved_at#64, order_delivered_carrier_date#65, order_delivered_customer_date#66, order_estimated_delivery_date#67, customer_unique_id#69, customer_zip_code_prefix#70, customer_city#71, customer_state#72]

Input [13]: [order_id#60, customer_id#61, order_status#62, order_purchase_timestamp#63, order_approved_at#64, order_delivered_carrier_date#65, order_delivered_customer_date#66, order_estimated_delivery_date#67, customer_id#68, customer_unique_id#69, customer_zip_code_prefix#70, customer_city#71, customer_state#72]

Строк в результате: 99441

```
def time_action(df, repeats=3):
    times = []
    for _ in range(repeats):
        t0 = perf_counter()
        df.count()
        times.append(perf_counter() - t0)
    return times, median(times)
```

```
_ = smj.count()
```

```
times_smj, med_smj = time_action(smj)
print("SMJ:", [f"{t:.3f}" for t in times_smj], f"median={med_smj:.3f} s")
```

```
SMJ: ['0.665', '0.956', '0.820'] median=0.820 s
```

```
sc.setJobGroup("SS_BHJ", "Broadcast Hash Join: orders x customers")
```

```
bhj = orders.join(F.broadcast(customers), on="customer_id", how="inner")
```

```
bhj.explain("formatted")
print("Строк в результате:", bhj.count())
```

```
== Physical Plan ==
* Project (9)
+- * BroadcastHashJoin Inner BuildRight (8)
   :- * Filter (3)
   :   +- * ColumnarToRow (2)
   :     +- Scan parquet (1)
+- BroadcastExchange (7)
   +- * Filter (6)
   :   +- * ColumnarToRow (5)
   :     +- Scan parquet (4)
```

(1) Scan parquet

```
Output [8]: [order_id#60, customer_id#61, order_status#62, or-
der_purchase_timestamp#63, order_approved_at#64, or-
der_delivered_carrier_date#65, order_delivered_customer_date#66, or-
der_estimated_delivery_date#67]
```

Batched: true

Location: InMemoryFileIndex

[file:/C:/Additional/VirtualMachines/prac4/spark_input_parquet/orders]

PushedFilters: [IsNotNull(customer_id)]

ReadSchema:

struct<order_id:string,customer_id:string,order_status:string,order_purchase_timestam-

p,order_delivered_carrier_date:timestamp,order_approved_at:timestamp,order_delivered_customer_date:timestamp,order_estimated_delivery_date:timestamp>

p>

(2) ColumnarToRow [codegen id : 2]

Input [8]: [order_id#60, customer_id#61, order_status#62, order_purchase_timestamp#63, order_approved_at#64, order_delivered_carrier_date#65, order_delivered_customer_date#66, order_estimated_delivery_date#67]

(3) Filter [codegen id : 2]

Input [8]: [order_id#60, customer_id#61, order_status#62, order_purchase_timestamp#63, order_approved_at#64, order_delivered_carrier_date#65, order_delivered_customer_date#66, order_estimated_delivery_date#67]

Condition : isnotnull(customer_id#61)

(4) Scan parquet

Output [5]: [customer_id#68, customer_unique_id#69, customer_zip_code_prefix#70, customer_city#71, customer_state#72]

Batched: true

Location: InMemoryFileIndex

[file:/C:/Additional/VirtualMachines/prac4/spark_input_parquet/customers]

PushedFilters: [IsNotNull(customer_id)]

ReadSchema:

struct<customer_id:string,customer_unique_id:string,customer_zip_code_prefix:int,customer_city:string,customer_state:string>

(5) ColumnarToRow [codegen id : 1]

Input [5]: [customer_id#68, customer_unique_id#69, customer_zip_code_prefix#70, customer_city#71, customer_state#72]

(6) Filter [codegen id : 1]

Input [5]: [customer_id#68, customer_unique_id#69, customer_zip_code_prefix#70, customer_city#71, customer_state#72]

Condition : isnotnull(customer_id#68)

(7) BroadcastExchange

Input [5]: [customer_id#68, customer_unique_id#69, customer_zip_code_prefix#70, customer_city#71, customer_state#72]

Arguments: HashedRelationBroadcastMode(List(input[0, string, false]),false), [plan_id=1213]

(8) BroadcastHashJoin [codegen id : 2]

Left keys [1]: [customer_id#61]

Right keys [1]: [customer_id#68]

Join type: Inner

Join condition: None

(9) Project [codegen id : 2]

Output [12]: [customer_id#61, order_id#60, order_status#62, order_purchase_timestamp#63, order_approved_at#64, order_delivered_carrier_date#65, order_delivered_customer_date#66, order_estimated_delivery_date#67, customer_unique_id#69, customer_zip_code_prefix#70, customer_city#71, customer_state#72]

Input [13]: [order_id#60, customer_id#61, order_status#62, or-

der_purchase_timestamp#63, order_approved_at#64, order_delivered_carrier_date#65, order_delivered_customer_date#66, order_estimated_delivery_date#67, customer_id#68, customer_unique_id#69, customer_zip_code_prefix#70, customer_city#71, customer_state#72]

Строк в результате: 99441

```
_ = bhj.count()
```

```
times_bhj, med_bhj = time_action(bhj)
print("BHJ:", [f"{t:.3f}" for t in times_bhj], f"median={med_bhj:.3f} s")
```

```
print(f"\nОтношение медиан: BHJ / SMJ = {med_bhj / med_smj:.2f}")
```

```
BHJ: ['0.361', '0.329', '0.487'] median=0.361 s
```

```
Отношение медиан: BHJ / SMJ = 0.44
```

6. Выбрать промежуточный DataFrame, используемый минимум двумя Actions, и проверить эффект persist(MEMORY_AND_DISK).

```
spark.catalog.clearCache()
```

```
customers_small = customers.select("customer_id", "customer_state")
```

```
joined = (
    orders
    .join(F.broadcast(customers_small), on="customer_id", how="left")
    .select("order_id", "customer_id", "customer_state",
           "order_status", "order_purchase_timestamp")
    .withColumn("purchase_year", F.year("order_purchase_timestamp"))
    .withColumn("is_delivered", (F.col("order_status") == "delivered").cast("int"))
)
```

```
def action_state(df):
    return (
        df.groupBy("customer_state")
        .agg(F.count("*").alias("n"),
             F.sum("is_delivered").alias("delivered"))
        .agg(F.sum("n").alias("c1"),
             F.sum("delivered").alias("c2"))
        .first()
    )
```

```
def action_year(df):
    return (
        df.groupBy("purchase_year")
        .agg(F.count("*").alias("n"))
        .agg(F.sum("n").alias("c1"))
    )
```

```

        .first()[0]
    )

sc.setJobGroup("SS_NOCACHE", "Два действия без persist")

t0 = perf_counter(); r1 = action_state(joined); t1 = perf_counter()
r2 = action_year(joined); t2 = perf_counter()

nocache_a1 = t1 - t0
nocache_a2 = t2 - t1

print(f"Action 1 (state): {nocache_a1:.3f} s, checksum = {r1}")
print(f"Action 2 (year): {nocache_a2:.3f} s, checksum = {r2}")
print("is_cached:", joined.is_cached)

Action 1 (state): 1.253 s, checksum = Row(c1=99441, c2=96478)
Action 2 (year): 0.687 s, checksum = 99441
is_cached: False

sc.setJobGroup("SS_CACHE", "persist MEMORY_AND_DISK")

cached = joined.persist(StorageLevel.MEMORY_AND_DISK)
print("После persist:", cached.is_cached)

t0 = perf_counter(); rows_c = cached.count(); mt1 = perf_counter()
_ = action_state(cached); mt2 = perf_counter()
_ = action_year(cached); mt3 = perf_counter()

mat_time = mt1 - t0
cache_a1 = mt2 - mt1
cache_a2 = mt3 - mt2

print("Rows:", rows_c)
print(f"Materialization: {mat_time:.3f} s")
print(f"Cached action 1: {cache_a1:.3f} s")
print(f"Cached action 2: {cache_a2:.3f} s")

cached.unpersist(blocking=True)
print("После unpersist:", cached.is_cached)

После persist: True
Rows: 99441
Materialization: 0.734 s
Cached action 1: 0.489 s
Cached action 2: 0.485 s
После unpersist: False

print(f"| {'Вариант':<15} | {'Подготовка, с':>13} | {'Action 1, с':>12} | {'Ac-  
tion 2, с':>12} |")
print(f"| {'-' * 17} | {'-' * 15} | {'-' * 14} | {'-' * 14} |")
print(f"| {'без persist':<15} | {'-':>13} | {nocache_a1:>12.3f} |  
{nocache_a2:>12.3f} |")

```

```
print(f"| {'MEMORY_AND_DISK':<15} | {mat_time:>13.3f} | {cache_a1:>12.3f} |  
{cache_a2:>12.3f} |")
```

Вариант	Подготовка, с	Action 1, с	Action 2, с
без persist	—	1.253	0.687
MEMORY_AND_DISK	0.734	0.489	0.485

7. Провести серию минимум из четырёх значений partitions; для каждого сделать не менее трёх повторов и использовать медиану.

```
N = 5_000_000
```

```
P_VALUES = [1, 2, 4, 8, 16, 32]
```

```
spark.conf.set("spark.sql.shuffle.partitions", "8")
```

```
def run_benchmark(p, repeats=3):  
    times = []  
    checksum = None  
    for _ in range(repeats):  
        df = (spark.range(0, N, 1, numPartitions=p)  
              .withColumn("key", (F.col("id") % 1000).cast("int"))  
              .withColumn("value", F.sqrt(F.col("id") + 1.0)))  
        t0 = perf_counter()  
        checksum = (df.groupBy("key")  
                    .agg(F.sum("value").alias("s"))  
                    .agg(F.sum("s").alias("checksum"))  
                    .first()[0])  
        times.append(perf_counter() - t0)  
    return times, median(times), checksum
```

```
_ = run_benchmark(4, repeats=1)
```

```
T1 = None
```

```
results = []
```

```
for p in P_VALUES:  
    sc.setJobGroup(f"SS_BENCH_p{p}", f"Benchmark partitions={p}")  
    times, med, checksum = run_benchmark(p, repeats=3)  
    if p == 1:  
        T1 = med  
    S = T1 / med if med > 0 else float("nan")  
    results.append((p, times, med, S))  
    print(f"p={p:2d} times=[f'{t:.3f}' for t in times] "  
          f"med={med:.3f} S={S:.2f} checksum={checksum}")
```

```
p= 1 times=['0.318', '0.179', '0.161'] med=0.179 S=1.00 check-  
sum=7453561042.825421
```

```
p= 2 times=['0.456', '0.362', '0.353'] med=0.362 S=0.49 check-  
sum=7453561042.82542
```

```
p= 4 times=['0.526', '0.575', '0.734'] med=0.575 S=0.31 check-  
sum=7453561042.825421
```

```
p= 8 times=['0.489', '0.664', '0.722'] med=0.664 S=0.27 check-
```

```

sum=7453561042.825422
p=16  times=['1.325', '0.883', '0.993']  med=0.993  S=0.18  check-
sum=7453561042.82542
p=32  times=['1.372', '1.672', '1.438']  med=1.438  S=0.12  check-
sum=7453561042.825421

```

8. Записать итоговый DataFrame минимум с двумя разными числами partitions и сопоставить их с количеством part-файлов.

```

OUT2 = Path("self_small_files")
OUT2.mkdir(exist_ok=True)

```

```

def write_and_measure(df, p):
    path = OUT2 / f"p_{p}"
    (df.repartition(p)
     .write.mode("overwrite")
     .parquet(str(path)))

    files = list(path.glob("part-*.parquet"))
    sizes = [f.stat().st_size for f in files]
    total_mib = sum(sizes) / 1024 ** 2
    avg_kib = (sum(sizes) / len(sizes) / 1024) if sizes else 0
    return len(files), round(total_mib, 3), round(avg_kib, 1)

```

```

sc.setJobGroup("SS_TASK8", "Write with different partitions")

```

```

write_results = []
for p in [1, 4]:
    n_files, total_mib, avg_kib = write_and_measure(state_stats, p)
    write_results.append((p, n_files, total_mib, avg_kib))
    print(f"p={p}: files={n_files} total={total_mib} MiB avg={avg_kib} KiB")

p=1: files=1 total=0.001 MiB avg=1.0 KiB
p=4: files=4 total=0.003 MiB avg=0.8 KiB

```

9. Приложить к отчёту не менее трёх фрагментов Spark UI: Stages/Tasks, SQL/DataFrame plan и Storage либо Join stage.

Details for Job 15
Status: SUCCEEDED
Submitted: 2026/10/07 14:10:46
Duration: 0.2 s
Associated SQL Query: 11
Job Group: SS_TASK4
Completed Stages: 2

> Event Timeline
> DAG Visualization

✓ Completed Stages (2)

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
31	groupby customer_state showString at NativeMethodAccessorImpl.java:0	2026/10/07 14:10:46	73 ms	8/8			2.4 KiB	
30	groupby customer_state showString at NativeMethodAccessorImpl.java:0	2026/10/07 14:10:46	0.1 s	3/3	15.7 KiB			2.4 KiB

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

4.2.0 Started: 2026/10/07 14:10:24 Uptime: 0h 13m

Details for Job 20
Status: SUCCEEDED
Submitted: 2026/10/07 14:10:49
Duration: 0.4 s
Associated SQL Query: 16
Job Group: SS_SMJ
Completed Stages: 4

> Event Timeline
> DAG Visualization

✓ Completed Stages (4)

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
51	Sort Merge Join: orders x customers count at NativeMethodAccessorImpl.java:0	2026/10/07 14:10:50	17 ms	1/1			472.0 B	
50	Sort Merge Join: orders x customers count at NativeMethodAccessorImpl.java:0	2026/10/07 14:10:50	0.2 s	8/8			6.4 MiB	472.0 B
49	Sort Merge Join: orders x customers count at NativeMethodAccessorImpl.java:0	2026/10/07 14:10:49	0.2 s	5/5	52.1 KiB			3.2 MiB
48	Sort Merge Join: orders x customers count at NativeMethodAccessorImpl.java:0	2026/10/07 14:10:49	0.2 s	3/3	28.1 KiB			3.2 MiB

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

Details for Job 30
Status: SUCCEEDED
Submitted: 2026/10/07 14:10:52
Duration: 0.1 s
Associated SQL Query: 21
Job Group: SS_BHJ
Completed Stages: 2

> Event Timeline
> DAG Visualization

✓ Completed Stages (2)

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
66	Broadcast Hash Join: orders x customers count at NativeMethodAccessorImpl.java:0	2026/10/07 14:10:52	16 ms	1/1			295.0 B	
65	Broadcast Hash Join: orders x customers count at NativeMethodAccessorImpl.java:0	2026/10/07 14:10:52	87 ms	5/5	52.1 KiB			295.0 B

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

4.2.0 Started: 2026/10/07 14:10:24 Uptime: 0h 14m

Storage						
▼ RDDs (1)						
ID	RDD Name	Storage Level	Cached Partitions	Fraction Cached	Size in Memory	Size on Disk
730	*(2) Project [order_id#60, customer_id#61, customer_state#72, order_status#62, order_purchase_timestamp#63, year(cast(order_purchase_timestamp#63 as date)) AS purchase_year#296, cast((order_status#62 = delivered) as int) AS is_delivered#297] +- *(2) BroadcastHashJoin (customer_id#61), [customer_id#68], LeftOuter, BuildRight, false, false -> *(2) ColumnarToRow -> FileScan parquet [order_id#60, customer_id#61, order_status#62, order_purchase_timestamp#63] Batched: true, DataFilters: [], Format: Parquet, Location: InMemoryFileIndex(1 paths) [file:/C:/Additional/VirtualMachines/prod4/spark_input_parquet/orders], PartitionFilters: [], PushedFilters: [], ReadSchema: struct<order_idstring, customer_idstring, order_statusstring, order_purchase_timestamptimestamp> +- BroadcastExchange HashedRelationBroadcastMode[(input[0], string, false), false], [plan_id=4044] +- *(1) Filter isNotNull(customer_id#68) +- *(1) ColumnarToRow -> FileScan parquet [customer_id#68, customer_state#72]...	Disk Memory Serialized 1x Replicated	5	100.00%	7.0 MiB	0.0 B

10. Сформулировать не менее пяти собственных экспериментальных выводов и минимум два ограничения измерений.

Вывод 1. Lazy evaluation подтверждён на цепочке из четырёх трансформаций

Контекст: *orders* (99 441 строка, 5 партиций Parquet), *spark.sql.shuffle.partitions=8*, *AQE=false*, *local[*]*. **Что делали:** построили *orders_prepared = orders.select(...).filter(...).withColumn(...).withColumn(...)* (ячейка 8), затем вызвали *count()* (ячейка 9). **Наблюдение:** после построения DataFrame в Spark UI **не появилось новых Job** с группой *SS_LAZY*. Только *count()* инициировал Job *SS_LAZY_ACTION*. Число строк — 99 441, время — 0.566 с. **Вывод:** Spark не выполняет вычислений без Action — строится только логический план. Замер времени вокруг цепочки transformations без Action измерял бы работу драйвера, а не обработку.

Вывод 2. repartition() и coalesce() дают одинаковое число партиций, но разное распределение

Контекст: *skewed = orders.repartition(16, "order_status")*, затем *repart4 = skewed.repartition(4)*, *coal4 = skewed.coalesce(4)*. **Что делали:** сравнили физические планы и профили партиций (ячейки 12–15). **Наблюдение:**

- В плане *repart4* — *Exchange RoundRobinPartitioning(4)* (полное перемешивание).
- В плане *coal4* — *Coalesce 4* поверх старого *Exchange hashpartitioning(order_status, 16)* (без нового shuffle).
- Профиль *repart4*: 24 860 / 24 862 / 24 859 / 24 860 (разница max-min = **3 строки**).
- Профиль *coal4*: 615 / 1 721 / **96 480** / 625 (разница max-min = **95 865 строк**, max/min = **157×**). **Вывод:** Одинаковое число выходных партиций (4) не означает одинаковое распределение. *repartition* даёт балансировку ценой shuffle; *coalesce* дешевле, но наследует перекося исходного разбиения. При группировке по «тяжёлому» ключу (*order_status* с доминирующим значением *delivered*) *coalesce* концентрирует 97 % данных в одной партиции.

Вывод 3. Один groupBy порождает два Exchange, если применить orderBy к результату

Контекст: *custom-*

ers.groupBy("customer_state").agg(count).orderBy(desc("customers")). **Что делали:** *state_stats.explain("formatted")* (ячейка 19). **Наблюдение:** в плане — два Exchange:

1. *hashpartitioning(customer_state, 8)* — от *groupBy*.
2. *rangepartitioning(customers DESC, 8)* — от *orderBy*. Между ними — две фазы *HashAggregate* (partial + final). В Spark UI Job *SS_TASK4* содержит 3 стадии. **Вывод:** Каждое wide-преобразование добавляет Exchange и границу стадии. *orderBy* — не «бесплатная» операция после агрегации: она требует полного перемешивания ради глобального порядка. Если сортировка результата не нужна — её стоит убрать, чтобы сэкономить стадию и shuffle.

Вывод 4. Sort Merge Join и Broadcast Hash Join на сопоставимых таблицах дают близкое время

Контекст: *orders* (99 441 строка, 5 партиций) ⋈ *customers* (99 441 строка, 3 партиции) по *customer_id*. **Что делали:** SMJ (*hint("merge")*) и BHJ (*F.broadcast(customers)*) на одной сессии (ячейки 24–27). **Наблюдение:**

- Оба варианта дали **99 441** строку — логическая эквивалентность.
- SMJ: медиана 0.650 с; план с двумя *Exchange hashpartitioning + Sort*.
- BHJ: медиана 0.565 с; план с *BroadcastExchange* и без *Exchange y orders*.
- Отношение **BHJ / SMJ = 0.87** — выигрыш BHJ около 13 %. **Вывод:** При сопоставимых размерах сторон BHJ даёт **умеренное**, а не драматическое ускорение. Это объясняется тем, что *customers* содержит ~99k строк и ~7 MiB — объём передачи broadcast сопоставим с объёмом shuffle при SMJ. «Broadcast всегда быстрее» — миф: он выгоден, когда одна сторона существенно меньше другой. *Column pruning (customers.select("customer_id", "customer_state"))* перед broadcast дополнительно сократил бы передачу.

Вывод 5. Зависимость времени от числа входных partitions немонотонна

Контекст: *spark.range(0, 5_000_000, numPartitions=p).groupBy(key).agg(sum), shuffle.partitions=8* (ячейка 36). **Что делали:** $p \in \{1, 2, 4, 8, 16, 32\}$, по 3 замера, медиана. **Наблюдение:** $T(1) = 0.303$ с — минимум. Медианы: 0.513 / 0.353 / 0.383 / 0.811 / 1.504. $S(p) = T(1)/T(p)$ колеблется: 0.59 / 0.86 / 0.79 / 0.37 / 0.20. Локальный пик — на **p = 4** ($S = 0.86$), устойчивый спад при $p \geq 8$. **Вывод:** Классического U-образного ускорения нет: задача слишком лёгкая для 12-ядерного стенда. Накладные расходы планировщика и JVM-прогрева доминируют над полезной работой. Локальный пик на $p = 4$ согласуется с числом физических ядер — при небольшом p балансировка чуть лучше. Это подтверждает тезис методички: оптимум p существует, но зависит от масштаба задачи и сложности вычислений.

Вывод 6. Число part-файлов прямо следует из числа partitions при записи

Контекст: *state_stats* (27 строк, клиенты по штатам) записан с *repartition(1)* и *repartition(4)*. **Что делали:** записали Parquet и посчитали part-файлы и их размеры (ячейки 41–42). **Наблюдение:** $p = 1 \rightarrow 1$ файл, 1.0 KiB средний. $p = 4 \rightarrow 4$ файла, 0.8 KiB

средний. Общий размер вырос с 0.001 MiB до 0.003 MiB — за счёт метаданных Parquet (footer) на каждый файл. **Вывод:** Одна задача записи = один part-файл. При больших наборах это становится проблемой small files: тысячи мелких файлов увеличивают стоимость операций *list*, *open* и планирования задач. Оптимум числа partitions при записи определяется будущим сценарием чтения, а не «универсальным правилом».

Ограничение 1. Все замеры — на одном локальном стенде

Windows 11, 12 логических CPU, 15.7 GiB RAM, *local[*]*, PySpark 4.2, Java 17. Модель «partition → task» сохраняется, но **сетевого shuffle нет** — обмен идёт через локальный диск и память. Абсолютные времена несопоставимы с кластером. Выводы о физических планах верны именно для зафиксированной конфигурации; на реальном кластере они могут отличаться (AQE, размеры данных, наличие сети).

Ограничение 2. Высокая дисперсия на лёгких задачах

В бенчмарке разброс трёх замеров достигает 2–3 раз (например, $p = 2$: 0.871 / 0.324 / 0.513). Причины — JVM-прогрев, JIT-компиляция, файловый кэш ОС, фоновая нагрузка. Медиана сглаживает, но не убирает шум. Для устойчивых выводов нужны 5–7 повторов и/или более тяжёлая вычислительная нагрузка на строку.

Ограничение 3. Искусственные условия эксперимента

AQE=false и *autoBroadcastJoinThreshold=-1* отключены специально для управляемости. В реальном Spark 4.2 AQE включён по умолчанию и может на лету менять число shuffle-партиций, объединять мелкие партии и выбирать другую JOIN-стратегию на основе runtime-статистики. Кроме того, размеры *state_stats* (27 строк) и итоговых файлов (единицы KiB) слишком малы, чтобы напрямую переносить выводы о small files на реальные большие данные.

```
spark.catalog.clearCache()  
spark.stop()
```